CogLean, straight through

Why prove, a theory proved, how, is it faithful, laws, experiments, and what is true

Jeff Shrager

October 2026

The problem

Cognitive theories are stated in prose, in simulation code, or as equations fitted to data.

- What follows from a theory's *core assumptions*, as distinct from its parameter settings or implementation details, is rarely checked.
- A simulation shows that a model *can* produce a result, with these parameters and these stimuli.
- It doesn't show which assumptions produced it, whether it survives other parameters, or whether a rival theory would produce it just as well.

Three kinds of evidence a theory can give

One run	**Every run**	**Every model satisfying the postulates**
one model, one parameter setting	one model, all its executions	all models the theory allows
the theory *can* do it	that model *always* does it	the postulates *entail* it
simulation	model checking, sweeps	proof

CogLean is built around the third kind, and keeps the other two in the same language.

Why Lean

- **One definition, run and proved.** The same definition of a memory can be *run* on an experiment's stimuli and *proved about* for all stimuli. The simulation and the theory can't drift apart.
- **Checked.** Every proof step is checked by a small trusted kernel.
- **General.** A theorem can quantify over all nets, all parameter values, all stimulus lists.
- **Cumulative.** Like Mathlib for mathematics: definitions made once, theorems imported rather than re-derived.

The guarantee is relative. Lean checks that a theorem follows from the definitions, not that the definitions capture the theory, still less that the theory is true of people.

Two theories, combined

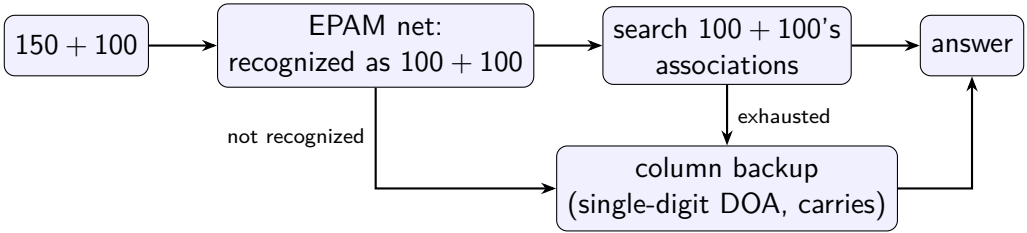
EPAM (Feigenbaum 1959)

- Stimuli are feature codes.
- A discrimination net sorts a code by testing features, down to a stored image.
- Learning grows a test only where two codes are confused, on the first difference noticed.
- So an unfamiliar code that agrees on every tested feature is *recognized as* a familiar one.

Distributions of associations (Siegler & Shrager 1984)

- Each problem has associations to answers, with strengths.
- Sample an answer in proportion to strength; state it if it clears a confidence criterion; try up to n times.
- Otherwise, use a backup strategy.
- Retrieval is used with probability $1 - (1 - R)^n$.

The multi-digit model



- Every postulate is a parameter: the encoding, the net, the strengths, the criterion, the search length, the backup.
- 18 predictions, each proved for every choice of them.

Confident errors

Let a familiar problem q have *sharp* associations (only its correct sum clears the criterion), and let x, with a different sum, be recognized as q. Then:

1. on q, whole-problem retrieval never errs;
2. on x, every whole-problem answer is wrong;
3. x uses the backup exactly as rarely as q does: $(1 - R_q)^n$.

In DOA alone, errors and backup use rise together. Confusion breaks the tie: a retrieved, wrong answer with no extra backup use.

A worked case

- Encoding: each digit in four bits (24 features). The net learns $100 + 100$ and $100 + 200$, which differ only in the second addend's hundreds digit, so that is all it tests.
- Associations: .90 on each problem's sum, .01 elsewhere; criterion .50; search length 1.
- $150 + 100$ is recognized as $100 + 100$.

$150 + 100$ answered 200, by retrieval	45/46
$150 + 100$ answered 250, by the columns	1/46

In words: Confident errors: the proof is a composition

Theorem (confident errors). Suppose:

- a familiar problem q calls up its own code for certain, and a stranger x calls up q's code for certain;
- q's associations are *sharp*: only q's correct answer clears the criterion;
- x's correct answer is different; no association set has total strength 0; the backups are proper distributions.

Then (1) q is never given a wrong whole answer; (2) every whole answer given to x is wrong; (3) x and q use the backup equally often.

Proof.

- (1) is the next lemma, applied to q itself.
- (2): on sharp associations every stated answer is q's correct answer, which is wrong for x.
- (3): both problems search the same associations, so each uses the backup with probability $(1 - R_q)^n$.

Confident errors: the proof is a composition

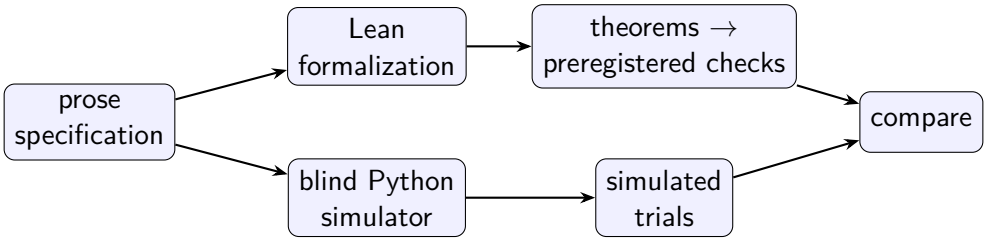
```
theorem confident_errors (correct : P → A) (x q : P) (hT : ∀ y, total as (str y) ≠ 0)
    (hbx : mass (backup x) = 1) (hbq : mass (backup q) = 1)
    (hq : retrieve (code q) = pure (some (code q)))
    (hx : retrieve (code x) = pure (some (code q)))
    (hs : Sharp str cc (code q) (correct q)) (hne : correct x ≠ correct q) :
    prob (recognizeRetrieve code retrieve str as cc n backup q) (wholeWrong correct q) = 0
    ↪ ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) (wholeWrong correct x) =
        prob (recognizeRetrieve code retrieve str as cc n backup x) isWhole ∧
      prob (recognizeRetrieve code retrieve str as cc n backup x) isBackup =
        prob (recognizeRetrieve code retrieve str as cc n backup q) isBackup := by
  refine ⟨confusion_gives_neighbours_answer code retrieve str as cc n backup correct q q
  ↪ (hT _) hq hs,
    wrong_eq_whole_of_sharp code retrieve str as cc n backup correct x _ _ (hT _) hx hs
    ↪ hne, ?_⟩
  rw [backup_prob_certain code retrieve str as cc n backup x _ hx hbx hT,
    backup_prob_certain code retrieve str as cc n backup q _ hq hbq hT]
```

What is checked, and what is not

```
#print axioms Cog.RR.confident_errors_of_laws
'Cog.RR.confident_errors_of_laws' depends on axioms: [propext, Classical.choice,
↪  Quot.sound]
```

- No `sorry` anywhere; only Lean's three standard axioms.
- Core Lean only: no Mathlib, so everything computes.
- **Not checked**: that the definitions say what EPAM, DOA or ACT-R say. That is what the prose specification, the blind simulation, and (planned) comparison with PyACTUp are for.

The idea: two independent readings



N-version programming (Avižienis 1985), applied to a theory. A disagreement means one of three things: the prose is ambiguous, the formalization drifted, or the simulator has a bug.

The procedure

1. **A prose specification** (`SPEC.md`): the model in plain words, with no Lean names, theorems, or predictions. Points the prose leaves open are listed as *open choices*.
2. **A preregistration** (`PREREG.md`): each theorem's observable statistic and how it is judged. Committed *before* the simulator existed; the implementer never saw it.
3. **A blind implementer**: a fresh AI agent, given only the specification, in an empty directory. Its tool calls were audited: it read only files it had written.
4. **Lean's exact distributions** for four shared instances.
5. **A harness** (`compare.py`): random instances, the preregistered checks, and a report.

Commits, 26 October 2026: specification and preregistration 19:16; simulator, unmodified, 19:21; harness and results 19:31.

Results: no failures

Check	Lean theorem	kind	tests	failures
Lean exact	`#eval MD.trial`	prob	243	0
P1	`columns_prob`	prob	394	0
P2	`whole_depends_only_on_route`	prob	8543	0
P3	`confusion_gives_neighbours_answer`	exact	200	0
P4	`confident_errors`	exact+prob	222	0
P7	`learning_preserves_trial`	exact	74692	0
P9	`columns_place_value`	exact	248800	0
P11	`discrimination_removes_confusion`	exact	91090	0
P6a, P6b, P14, P17, P18			0	*vacuous*

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

Finding 1: five predictions were never tested

- Predictions 6a, 6b, 14, 17 and 18 need the net's path to test *every* feature of a digit or a column: 4 bits, 9 thresholds, or 100 fact features.
- EPAM grows one test per confusion, so paths are short. In no learned net did the hypothesis ever hold.
- The theorems are true. But they say almost nothing about the nets EPAM actually learns.

Lesson. A theorem can be correct and nearly irrelevant to the models the theory produces. Restate it in terms of what learned nets do test: for example, "the tested bits of a digit restrict it to a set".

Finding 3: a prediction depends on an open choice

The predicted learning sequence: a confident error, then column work, then fluent correct retrieval.

O2: new problem starts with	discriminated	canonical	settles on correct	mixed	trapped on error
no associations	48	40	7	1	0
only the correct answer	61	28	33	0	0
a copy of the neighbour's	44	0	2	35	7

- With no associations, the predicted sequence appears (40 of 48).
- Starting with the correct answer, the column phase is often skipped.
- Copying the neighbour's associations perpetuates the error.

The generality problem

There are a great many specific models: EPAM, ACT-R, SAM, REM, MINERVA 2, TODAM, connectionist networks...

- Formalizing them one at a time doesn't scale.
- A pile of formalized models doesn't say what any two of them *disagree* about.
- And none of them is "true": unlike mathematics, there's no settled foundation to build on.

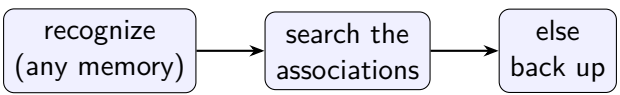
The same problems, two memories

	EPAM	ACT-R
$150 + 100$ answered 200 (the neighbour's sum)	45/46	225/529
$150 + 100$ answered 300 (differs on a tested digit)	0	225/1058
$150 + 100$ answered correctly	1/46	383/1058
errors on the practiced $100 + 100$	0	225/782
the same, after also studying $100 + 300$	0	3375/5842

ACT-R: confusion is graded, nothing is retrieved for certain, and new practice interferes with old.

The composition, extracted

The model's trial is one case of a general composition, now in the shared library:



- Over any problems and any answers, with "the correct answer" a parameter.
- Confident errors, and the rest, are proved once for the composition.
- Multi-digit addition is an instance: one bridge lemma carries every probability across, and its theorems keep their names.

A second task: naming people

Faces are codes; familiar Ann (000) and Bob (110); stranger Carl (100), one feature from each. The backup is "don't know".

	EPAM	ACT-R
Carl named Bob	45/46	225/391
Carl named Ann	0	225/782
Carl: "don't know"	1/46	107/782
Bob: "don't know"	1/46	229/2254
Bob named Ann	0	225/2254

With EPAM, *confident misnaming* is inherited from the composition's theorem, for every search length. No arithmetic anywhere.

Six laws

Certainty	An item just studied is retrieved by itself with probability 1.
Stability	Studying z leaves the chance that y retrieves itself unchanged.
Order-freedom	Swapping the last two study events leaves retrieval unchanged, for every probe.
Recency	Of two new items studied last, a probe equally similar to both retrieves the later more often.
Stored-only	Nothing is retrieved that was never studied.
Determinacy	Every retrieval has a single outcome.

Four families

- **Discrimination** (EPAM): a net of feature tests; learning grows a test where codes are confused.
- **Activation** (ACT-R declarative memory): each use adds activation that decays; partial matching; a failure weight. Decay 1, temperature 1: exactly rational.
- **Strength with a criterion** (after DOA): strengths only add; sample by strength, discounted by mismatch; state if above criterion.
- **Global matching** (MINERVA 2): every study stores a trace; a probe activates all traces by the cube of similarity; memory returns the *echo*, their weighted blend.

The matrix

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
Discrimination	**has**	**has**	lacks	lacks	**has**	**has**
Activation	lacks	lacks	lacks	**has**	**has**	lacks
Strength	lacks	lacks	**has**	lacks	**has**	lacks
Global matching	lacks	lacks	**has**	lacks	lacks	**has**

- **has**: a general proof, for every parameter choice meeting stated conditions.
- lacks: a counterexample, computed exactly on the eight 3-bit codes.
- Every law is had by some family and lacked by another; every family has its own row. A script checks both.

The laws were tested, and failed once

- The first four laws were chosen with three families in view.
- Global matching was added as a test. On those four laws, it had *exactly* the row of strength memory.
- So the laws missed something. What is distinctive about global matching? Its output is a blend.
- Study 110, 101, 011; probe with 111 (their prototype, never studied): MINERVA returns 111.
- Hence the stored-only law, which separates them.

Rule: a law enters only with a witness on each side.

From laws: which memories give confident errors?

Theorem. Any memory with three laws (*determinacy*, *certainty*, *stability*) produces confident errors whenever a stranger can be taken for a studied item with sharp associations.

- In the matrix of memory families, only discrimination (EPAM) has all three.
- Face naming's confident misnaming is derived this way, from EPAM's matrix cells plus one computed fact (Carl can be taken for Bob).
- The evidence judges that people lack certainty. So explaining confident errors in people needs a graded version: these laws with high probability, not probability 1.

Reversing the workflow

Usually: choose an experiment, compute each theory's predictions.

Instead:

1. find two laws that disagree;
2. derive a design on which they make incompatible predictions;
3. prove the separation, for every memory with either law;
4. state the response model that connects it to what participants say.

Recency against order-freedom

Design. Two new items a, b; a probe x equally similar to both. Condition AB studies a then b; condition BA studies b then a; both probe with x.

$$D = P(\text{retrieve } b) - P(\text{retrieve } a), \qquad \Delta = D_{AB} - D_{BA}.$$

Theorems, for any memory:

- order-freedom $\Rightarrow \Delta = 0$;
- recency $\Rightarrow \Delta > 0$;
- so no memory has both laws, wherever such a design exists.

From retrieval to report

Forced choice: report A if a is retrieved, B if b is; otherwise guess B with probability γ.

$$\Delta_{\text{report}} = \Delta + (2\gamma - 1)\,(o_{AB} - o_{BA}),$$

where o is the chance of retrieving neither item.

- Order-freedom: $\Delta_{\text{report}} = 0$, for every γ.
- Recency: $\Delta_{\text{report}} > 0$ with unbiased guessing, or when "neither" is equally likely in both orders.
- A biased guesser can mask or reverse the recency prediction.

The same design, four memories

Memory	Δ, no prior item	Δ, after 000	why
ACT-R	$10/17$		recency, so $\Delta > 0$
Strength	0	0	order-free
MINERVA	0	0	order-free
EPAM	0	-2	neither law: computed

- ACT-R's reported effect is $10/17$ for every guessing bias: both orders leave the same chance of "neither".
- EPAM shows the most extreme *primacy* possible. After 000, whichever item is learned first is discriminated from it, and the probe always retrieves that one. Its order effects come from the shape of the net, not from time.

What is true here?

In mathematics, a proved theorem is true. Here there are only competing theories. What can a library build on?

Claim	Example	Status
A law entails a prediction	stability $\Rightarrow$ practiced items stay practiced	proved, permanent
A family has a law	discrimination nets have stability	proved, permanent
People have a law	people lack stability in recall	empirical, revisable

Which theory is right follows from the second and third rows together.

Against the evidence

	Certainty	Stability	Order-free	Recency	Stored-only	Determinate
People (judged)	*lacks*	*mixed*	*lacks*	*has*	*mixed*	*mixed*

- A judgment, not a theorem, with sources, kept apart from the proofs.
- Each entry states its **bridge**: how a measure (a report) is taken to show a law (about retrieval).
- Activation fits every settled entry, but each fit rests on unformalized bridges.

Where this is weaker than mathematics

- **Definitions are contestable.** A family is one formalization of a theory. We check that link with blind simulations and community implementations.
- **Laws are idealizations** and presuppose a representation (binary codes, feature-count similarity).
- **Families are flexible.** The informative facts are parameter regions, not single counterexamples.
- **Evidence is never final.** The empirical row will change; the proved rows won't, and the matrix says at once what follows.

Next

- **Learning across trials**: feedback updates memory, so memory states become random; multi-digit learning sequences are the target.
- **Graded laws**, and the parameter regions of the "lacks" cells.
- **More families as tests**: REM, a connectionist memory, CHREST, ACT-R with blending.
- **Observational semantics**: decision and report components, so laws about retrieval predict what participants say.
- **Beyond memory**: strategy choice, then production systems.

Take-home

- Cognitive theories can be stated so that their consequences are *proved*, for every parameter setting.
- The durable content is conditional: laws entail predictions; families have or lack laws.
- A matrix of families against laws tells you which experiments are worth running, and what a result refutes.
- The value is not just that EPAM, ACT-R and others can each be formalized, but that results about one component can be combined with results about another to give predictions none of them anticipated.

github.com/jeffshrager/CogLean